Find the Promos: an AI Promo Discovery Engine for Microdramas

Problem 1 ("Find the Promos") · English US pack "I Married a Broken Soldier. He was a Duke" · 50 episodes, timestamped screenplays · free tiers only · prototype & all raw outputs linked in the submission form

1 · WHY THIS PROBLEM, WHERE I DREW THE LINE

Microdrama retention is won or lost on a 20–60 s promo: one cut that stops the scroll, sells the stakes, and makes a cold viewer need the next episode. Today editors pick those by hand, show by show, week by week — a repeatable, judgment-heavy job worth an AI first pass.

I scoped to *selection only*: find candidate clips and rank them by ad-worthiness. No editing, music or overlays — none of that was asked for. **Assumption (assignment fine print):** the pack is a show Kuku will release; screenplay timestamps are ground truth; the 1,487 dialogue events are the natural promo anchors; multi-segment promos are *schema-supported but not benchmarked* (all shipped candidates are contiguous).

2 · THE APPROACH — TWO MODES, ONE FUNNEL

```
input pack → Parser v2 → Context Builder → Fast Discovery → BLIND LLM Judge → Promo Ranker
            3,727 events   399 windows      84 union-84    53 accept/84    slate + reserve
```

Mode 1 — Fast Discovery (retrieval, high recall). Candidates built from screenplay events, embedded with `google/embeddinggemma-2` (270M-effective text tower, 768-dim, float32 CPU, text-only load, MRL renormalise-on-truncate). Document side: full chronological dialogue (speakers + deliveries) + camera/action/sound lines, as a titled document. Query side: the model's documented `SearchQuery` instruction, 13 categories (10 drama + 3 structural). **No similarity threshold** — an arbitrary cut would discard the boundary cases the judge exists for. Merge-top-K per category, dedup by candidate_id while retaining each candidate's complete similarity row.

Mode 2 — Contextual LLM Judge (selection). A blind judge (staged 34.6B Q4 coder model via local llama.cpp, `enable_thinking=false`, temp 0.0) sees *only* the candidate + its ±12 s narrative context and the episode logline/continuity notes — never retrieval scores. Rubric: opening hook 25% · cliffhanger 25% · standalone clarity 20% · narrative progression 15% · emotional impact 15%. **Every decisive judgement must cite the payload's real event_ids, machine-checked — invented ids cannot survive validation.** Outright rejection is a first-class result; 31/84 were rejected. Judge and evaluator are the same stage (no extra tuning); the Kuku interview can connect stronger models by swapping one provider line.

3 · ACTUAL RESULTS (RAW, HONEST, REPRODUCIBLE)

Stage & metric	Value	Reading
Parser → candidates	50 eps / 100 scenes / 3,727 events → 399 windows	1 warning kept verbatim (ep 22 sound-FX overflow); 4–15 windows/episode
Fast Discovery union	84/399 (21.1%) from 130 slots; 46 duplicates merged	recall-first; nothing below-K is dropped silently
Judge pass	84/84 ok, 53 accept / 31 reject; mean 3.43; median 3.55; max 4.4	strictest axis: standalone_clarity (3.06) — the paid-ad gate
Judge vs embedding rank	Spearman ρ ≈ **0.04** · Jaccard@10 = **5.3%**	two orthogonal lenses; result kept *because* they disagree
Human-reviewed precision (top-20, independent sample)	Precision@5 = **1.00** · @10 = **1.00** · @20 = **1.00**	33-pair sample; 16 strong + 4 usable in judged top-20; 30/33 judge↔reviewer agreement
Failure the judge *caught*	Ep35 `807f9f` backstory monologue (embed rank #1)	rejected — dead-air opening; slow-burn monologues are never ads
Cluster the judge *trusts communally*	Ep1 wedding-abandonment arc	top-ranked slate (rank 1 `39a710` → 4.4; rank 4 `91e509` → 4.2)
Ep33 coerced-testimony	embed ranked #1; judge kept some, rejected others	menace-as-promo can work — judge stays comparatively permissive when standalone is clean
Final ranking	49-slate → top-12 featured + 7 reserve (featured cap 2 per (episode,arc)); no arc holds more than 2 slots	6 near-duplicate clusters collapsed to their strongest head
Compute on this 8-core CPU	~65 minutes accumulated judge latency (≈46 s/candidate), 4.1 s warm cache re-run; 454,662 + 91,399 tokens	submission kept an LLM; judge calls ≈ free locally, paid tier would cost ≈ one $0.15 run

4 · WHERE IT FAILS (THE FIVE LENSES: QUALITY CHECKING)

1. **Coercion's edge cases:** ep33 coercion beats passed the threshold by judgment while similar ones did not; standalone clarity is the swing factor and it's genuinely borderline — a catalog buyer may disagree with both my reviewer and the judge. Recorded, not hidden.
2. **Cliffhanger policy vs taste:** judge rejected every clip ending on calm resolutions — correct for ad testing, but a warm brand might prefer them. Two "usable-but-rejected" (ep47 revenge-labor, ep48 Catherine's threat) are flagged as reviewer-contested.
3. **Arc-cap ≠ comedy:** cap is policy, not proof; the reserve list preserves the displaced, ready to reinstate with one flag.

4. **One model, one prompt:** results are `kuku-judge-v1.0.0` + one 34.6B coder model; an instruct chat model may reshuffle mid-tier verdicts at the interview (one-line provider swap, no pipeline change).
5. **Structural near-dupes survive:** normalised video clips at the 4fps layer are invisible to us; two hooks from the same arc class will still sometimes co-qualify. Diversity handles it on average, not per pair.

5 · SCALE & HOW TO RUN

Every show, every week. Parser+builder are cache-linear in the number of episodes; retrieval re-embeds only changed candidates (content-addressed cache validated by a 198-test suite); judge calls run once per changed candidate; a fresh 50-episode full-pack run is an ≈2 h batch, and re-screening interactively is ~4 s warm. Multi-episode/cross-episode promos are supported in the representation (schema has per-segment windows and a `segments` array) and are the first thing I'd enable next, behind a new validation pass.

Commands:

```
python script_parser.py --input pack.zip --output
parsed_output/
python context_builder.py --input parsed_output/
parsed_episodes.json
python candidate_retriever.py --candidates context_output/
candidates.json
python llm_judge.py --retrieval-results retrieval_output/
fast_discovery_results.json
python promo_ranker.py && python final_recommendations.py
```

Python 3.10+ stdlib + numpy; ~2 GB model download (HF, once); no external API keys. Raw outputs, rejection log, human labels ship with the form (Kuku-confidential, not public git).

6 · WHY I TRUST IT (VALIDATION & AI-USAGE HONESTY)

Determinism is guarded by unit/integration tests per module, byte-compared regression runs, and fixed seeds; the LLM pass runs once and its output is validated, not nudged. AI was used end-to-end as an engineering pair (opencode + agent tooling on Python 3.14). **Where the AI got it wrong, measurably:** EmbeddingGemma-2 rated ep33's *coerced-testimony* scenes the #1 “betrayal” material; both the blind LLM judge and my independent human pass re-ranked/rejected them — had we trusted retrieval alone, the slate would have opened with scenes no buyer wants. Second mistake: the first judge pass crashed at 15 min, discarding in-memory judge state — fixed by streaming a per-candidate cache to disk, which made the submission's own retry survivable.

A `models/` folder of 1.55 GB is intentionally excluded from the repo; it is reproducible with one command (README §models). Parser v2, Context Builder v1 and Stage-3 retrieval remain untouched by this submission — frozen exactly as approved at Stages 1–3.